

# Minor notes on merge

I wanted to share minor notes on the use of `merge` in pandas, when it comes to merging `DataFrames` of different structures.

We may be merging `DataFrames` where one has repeated measurements of the participants and another has a single row for each participant.

```
In [1]: # Toy example

# Repeated measurements (Four timepoints: m01, m02, m03, m04)
#
# +-----+-----+-----+-----+
# | PID   | timepoint | variable1 | variable2 |
# +-----+-----+-----+-----+
# | A001 | m01      | xx.xxx   | yy.yyy   |
# +-----+-----+-----+-----+
# | A001 | m02      | xx.xxx   | yy.yyy   |
# +-----+-----+-----+-----+
# | A001 | m03      | xx.xxx   | yy.yyy   |
# +-----+-----+-----+-----+
# | A001 | m04      | xx.xxx   | yy.yyy   |
# +-----+-----+-----+-----+
# | A002 | m01      | xx.xxx   | yy.yyy   |
# +-----+-----+-----+-----+
# | A002 | m02      | xx.xxx   | yy.yyy   |
# +-----+-----+-----+-----+
# | ...  | ...      | ...      | ...      |
# +-----+-----+-----+-----+
# | A003 | m04      | xx.xxx   | yy.yyy   |
# +-----+-----+-----+-----+

import numpy as np
import pandas as pd

np.random.seed(42)

df_A = pd.DataFrame({'ID': ['A001']*4 + ['A002']*4 + ['A003']*4,
                     'timepoint': ['m01', 'm02', 'm03', 'm04']*3,
                     'variable1': np.random.random(12),
                     'variable2': np.random.random(12)+1})
```

```
In [2]: # Another DataFrame with single entry for each participant
#
# +-----+-----+-----+-----+
# | PID   | variable3 | variable4 | variable5 |
# +-----+-----+-----+-----+
```

```
# | nan | zz.zzz | aa.aaa | bb.bbb |
# +-----+-----+-----+-----+
# | A002 | zz.zzz | aa.aaa | bb.bbb |
# +-----+-----+-----+-----+
# | A003 | zz.zzz | aa.aaa | bb.bbb |
# +-----+-----+-----+-----+
np.random.seed(36)

df_B = pd.DataFrame({'ID': [np.nan] + ['A002'] + ['A003'],
                     'variable3': np.random.random(3),
                     'variable4': np.random.random(3),
                     'variable5': np.random.random(3)})
```

```
In [ ]: # We can merge using the column 'PID',
# because that's present in both DataFrames.

# Merging the Left(df_A) and the Right(df_B) DataFrames,
# using the 'ID' column present in both df_A and df_B.
merged = pd.merge(df_A, df_B, on='ID')

# This is equivalent to
# - `merged = df_A.merge(df_B, on='ID')`
# - `merged = df_A.merge(df_B, left_on='ID', right_on='ID')`

# `pd.merge()` assumes arguments in this order:
#   `pd.merge(left=, right=, ...)`

# So this means that the first DataFrame you provide to
# `pd.merge()` function (ex. `df_A`) will be recognized as
# the value for the argument `left`.

# Now if you directly use the `.merge` method of `pd.DataFrame`,
# the DataFrame onto which a new DataFrame will be merged using
# `merge` method will be the `left` DataFrame.
```

`how` was what I meant by `method` previously... so here you could decide the manner you merge different DataFrames.

- `'inner'` -> **intersection** (only matching keys; default option)
- `'left'` -> all keys from the **left DataFrame**
- `'right'` -> all keys from the **right DataFrame**
- `'outer'` -> **union** (all keys from both, filling missing with NaN)

```
In [4]: # Check the output of `how='inner'`
merged
```

Out [4]:

|   | ID   | timepoint | variable1 | variable2 | variable3 | variable4 | variable5 |
|---|------|-----------|-----------|-----------|-----------|-----------|-----------|
| 0 | A002 | m01       | 0.156019  | 1.304242  | 0.601614  | 0.807359  | 0.259151  |
| 1 | A002 | m02       | 0.155995  | 1.524756  | 0.601614  | 0.807359  | 0.259151  |
| 2 | A002 | m03       | 0.058084  | 1.431945  | 0.601614  | 0.807359  | 0.259151  |
| 3 | A002 | m04       | 0.866176  | 1.291229  | 0.601614  | 0.807359  | 0.259151  |
| 4 | A003 | m01       | 0.601115  | 1.611853  | 0.952181  | 0.363215  | 0.629762  |
| 5 | A003 | m02       | 0.708073  | 1.139494  | 0.952181  | 0.363215  | 0.629762  |
| 6 | A003 | m03       | 0.020584  | 1.292145  | 0.952181  | 0.363215  | 0.629762  |
| 7 | A003 | m04       | 0.969910  | 1.366362  | 0.952181  | 0.363215  | 0.629762  |

A001 is gone, because the entry is only in `df_A`, but not in `df_B`. If you want to spare A001, you would *prioritize* the DataFrame that has that entry. If you make it the **left DataFrame**, you would use `how='left'`.

```
In [5]: merged_left = pd.merge(df_A, df_B, on='ID', how='left')
merged_left
```

Out [5]:

|    | ID   | timepoint | variable1 | variable2 | variable3 | variable4 | variable5 |
|----|------|-----------|-----------|-----------|-----------|-----------|-----------|
| 0  | A001 | m01       | 0.374540  | 1.832443  | NaN       | NaN       | NaN       |
| 1  | A001 | m02       | 0.950714  | 1.212339  | NaN       | NaN       | NaN       |
| 2  | A001 | m03       | 0.731994  | 1.181825  | NaN       | NaN       | NaN       |
| 3  | A001 | m04       | 0.598658  | 1.183405  | NaN       | NaN       | NaN       |
| 4  | A002 | m01       | 0.156019  | 1.304242  | 0.601614  | 0.807359  | 0.259151  |
| 5  | A002 | m02       | 0.155995  | 1.524756  | 0.601614  | 0.807359  | 0.259151  |
| 6  | A002 | m03       | 0.058084  | 1.431945  | 0.601614  | 0.807359  | 0.259151  |
| 7  | A002 | m04       | 0.866176  | 1.291229  | 0.601614  | 0.807359  | 0.259151  |
| 8  | A003 | m01       | 0.601115  | 1.611853  | 0.952181  | 0.363215  | 0.629762  |
| 9  | A003 | m02       | 0.708073  | 1.139494  | 0.952181  | 0.363215  | 0.629762  |
| 10 | A003 | m03       | 0.020584  | 1.292145  | 0.952181  | 0.363215  | 0.629762  |
| 11 | A003 | m04       | 0.969910  | 1.366362  | 0.952181  | 0.363215  | 0.629762  |

If you provide `df_A` as the **right DataFrame**, then use `how='right'`. Think of `left` as your reference table - you're saying "keep all of this, and add what matches from the other side."

```
In [ ]: merged_right = pd.merge(df_B, df_A, on='ID', how='right')
# Shuffling columns to make it look more like above
# (F string and list comprehension used to type less)
merged_right = merged_right[['ID', 'timepoint'] + \
                             [f'variable{i}' for i in range(1, 6)]]
merged_right
```

```
Out [ ]:
```

|    | ID   | timepoint | variable1 | variable2 | variable3 | variable4 | variable5 |
|----|------|-----------|-----------|-----------|-----------|-----------|-----------|
| 0  | A001 | m01       | 0.374540  | 1.832443  | NaN       | NaN       | NaN       |
| 1  | A001 | m02       | 0.950714  | 1.212339  | NaN       | NaN       | NaN       |
| 2  | A001 | m03       | 0.731994  | 1.181825  | NaN       | NaN       | NaN       |
| 3  | A001 | m04       | 0.598658  | 1.183405  | NaN       | NaN       | NaN       |
| 4  | A002 | m01       | 0.156019  | 1.304242  | 0.601614  | 0.807359  | 0.259151  |
| 5  | A002 | m02       | 0.155995  | 1.524756  | 0.601614  | 0.807359  | 0.259151  |
| 6  | A002 | m03       | 0.058084  | 1.431945  | 0.601614  | 0.807359  | 0.259151  |
| 7  | A002 | m04       | 0.866176  | 1.291229  | 0.601614  | 0.807359  | 0.259151  |
| 8  | A003 | m01       | 0.601115  | 1.611853  | 0.952181  | 0.363215  | 0.629762  |
| 9  | A003 | m02       | 0.708073  | 1.139494  | 0.952181  | 0.363215  | 0.629762  |
| 10 | A003 | m03       | 0.020584  | 1.292145  | 0.952181  | 0.363215  | 0.629762  |
| 11 | A003 | m04       | 0.969910  | 1.366362  | 0.952181  | 0.363215  | 0.629762  |

`indicator=True` reveals which rows were matched by preparing a new column `_merge`.

```
In [7]: merged = pd.merge(df_A, df_B, on='ID', how='left', indicator=True)
merged
```

Out[7]:

|    | ID   | timepoint | variable1 | variable2 | variable3 | variable4 | variable5 | _me    |
|----|------|-----------|-----------|-----------|-----------|-----------|-----------|--------|
| 0  | A001 | m01       | 0.374540  | 1.832443  | NaN       | NaN       | NaN       | left_c |
| 1  | A001 | m02       | 0.950714  | 1.212339  | NaN       | NaN       | NaN       | left_c |
| 2  | A001 | m03       | 0.731994  | 1.181825  | NaN       | NaN       | NaN       | left_c |
| 3  | A001 | m04       | 0.598658  | 1.183405  | NaN       | NaN       | NaN       | left_c |
| 4  | A002 | m01       | 0.156019  | 1.304242  | 0.601614  | 0.807359  | 0.259151  | b      |
| 5  | A002 | m02       | 0.155995  | 1.524756  | 0.601614  | 0.807359  | 0.259151  | b      |
| 6  | A002 | m03       | 0.058084  | 1.431945  | 0.601614  | 0.807359  | 0.259151  | b      |
| 7  | A002 | m04       | 0.866176  | 1.291229  | 0.601614  | 0.807359  | 0.259151  | b      |
| 8  | A003 | m01       | 0.601115  | 1.611853  | 0.952181  | 0.363215  | 0.629762  | b      |
| 9  | A003 | m02       | 0.708073  | 1.139494  | 0.952181  | 0.363215  | 0.629762  | b      |
| 10 | A003 | m03       | 0.020584  | 1.292145  | 0.952181  | 0.363215  | 0.629762  | b      |
| 11 | A003 | m04       | 0.969910  | 1.366362  | 0.952181  | 0.363215  | 0.629762  | b      |

So What I'd recommend is to first think about the reference DataFrame that you will *prioritize*, probably the one with the greatest number of unique 'ID', because you may want to save as many *datapoints* as possible to begin with and then gradually trim down. Then you can continue **merging** other DataFrames to this one, where the prioritized DataFrame will be the **left** in `pd.merge()`.

One more thing to note is that values of columns: **variable3**, **variable4**, and **variable5** repeat by the number of **timepoint** rows (ex. `m01`, `m02`, `m03`, and `m04`) associated with unique **ID** (ex. `A001`). This is an expected behavior (*one-to-many* or *many-to-one* join).

Each participant in `df_B` has one row, but in `df_A` there are multiple timepoints. Hence each `df_B` row gets duplicated for every match in `df_A` (*one-to-many*).